Voice-Based Concept Understanding Analyser

Team Member:
Sowmya Anala - sowmyaanala5@gmail.com

Demo Link:
https://drive.google.com/file/d/1saE4wKLMRPcP2fTpI3Apo6yFjrErrKVf/view?usp=sharing

Project Description:

Voice-Based Concept Understanding Analyser is an AI-powered application that evaluates a user's conceptual understanding by analyzing spoken explanations. Instead of assessing only speech fluency, the system focuses on determining how well a user understands and explains a given concept through voice input.

The application accepts an audio recording, converts speech into text using the OpenAI Whisper speech recognition model, and analyzes the extracted transcript using Sentence-BERT for semantic similarity evaluation. The system compares the spoken explanation with a reference concept, calculates a semantic similarity score, and generates an understanding score along with meaningful feedback.

The application also provides an audio waveform visualization using Librosa and Matplotlib, enabling users to inspect their recorded speech. Finally, a detailed PDF report is generated using ReportLab, summarizing the transcript, similarity score, concept understanding level, and performance feedback.

Built using Python and Streamlit, the application offers an intuitive and interactive interface that enables students, educators, and professionals to evaluate conceptual knowledge through voice-based assessments. The project demonstrates the integration of Speech Recognition, Natural Language Processing (NLP), Semantic Analysis, and Report Generation into a single intelligent learning platform.

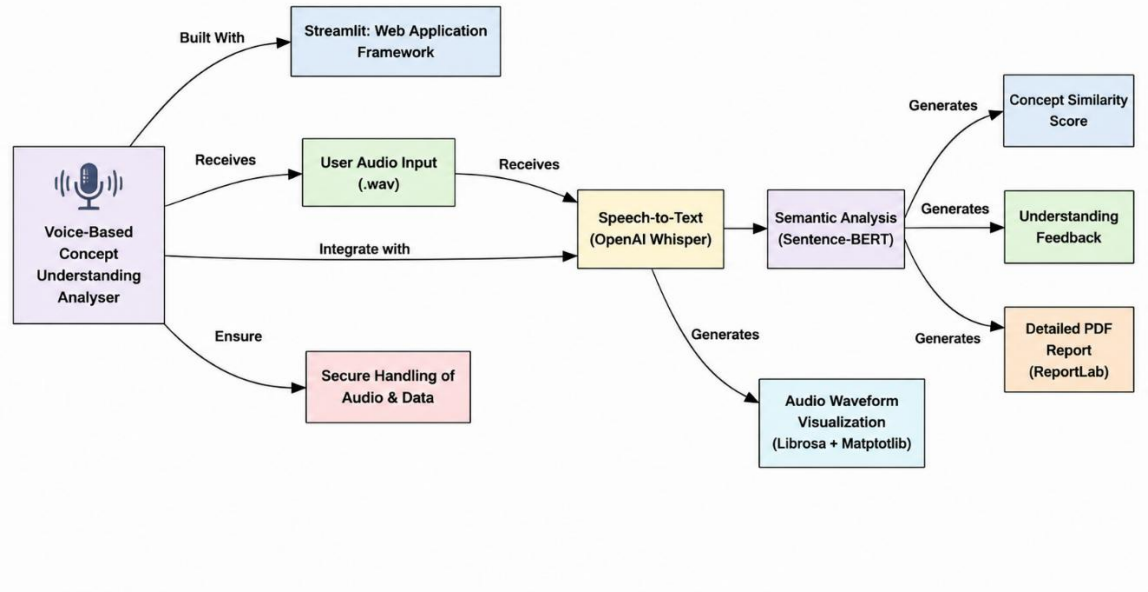
Scenarios

Scenario 1: Student Learning Assessment
A student records an explanation of a programming concept such as Object-Oriented Programming. The application converts the speech into text, compares the explanation with the expected concept, and generates a semantic similarity score along with personalized feedback to help the student identify areas for improvement.

Scenario 2: Interview Preparation
A job applicant practices answering technical interview questions by speaking into the application. The system evaluates how accurately the concepts are explained, provides an understanding score, and generates a detailed performance report that helps improve interview readiness.

Scenario 3: Self-Learning and Concept Revision
A learner studying online records explanations of various academic topics to test their understanding. The application analyzes the spoken response, measures concept accuracy using semantic similarity, visualizes the audio waveform, and produces a downloadable PDF report containing the transcript, concept score, and improvement suggestions

Technical Architecture:



Description:

The Voice-Based Concept Understanding Analyser follows a modular AI-driven architecture that integrates speech recognition, natural language processing, semantic analysis, visualization, and report generation into a unified workflow. The user uploads a speech recording through a Streamlit-based web interface. The audio is preprocessed using Librosa and converted into text using the OpenAI Whisper speech recognition model. The generated transcript is then analyzed using Sentence-BERT to compare the spoken explanation with a reference concept and compute a semantic similarity score. A scoring engine evaluates the user's conceptual understanding and generates personalized feedback. The application also creates an audio waveform visualization and compiles all results into a comprehensive PDF report using ReportLab. This modular architecture ensures accuracy, scalability, and an intuitive user experience for concept-based learning and assessment.

Pre-requisites:

- Python 3.10 or above
- Streamlit Framework
- OpenAI Whisper
- Sentence-BERT
- Librosa
- Matplotlib
- ReportLab
- FFmpeg
- Visual Studio Code
- Git and GitHub (optional)

Project Workflow:

Milestone 1: Speech Recognition Model Selection and System Architecture

- **Activity 1.1:** Select and integrate the OpenAI Whisper model for Speech-to-Text conversion.
- **Activity 1.2:** Select the Sentence-BERT model for semantic similarity and concept evaluation**.**
- **Activity 1.3:** Define the system architecture, including the Streamlit interface, speech recognition, semantic analysis, scoring, and report generation modules.
- **Activity 1.4:** Set up the development environment by installing Python, Streamlit, Whisper, Sentence-BERT, Librosa, Matplotlib, ReportLab, and FFmpeg.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the Speech-to-Text module using OpenAI Whisper.
- **Activity 2.2:** Implement semantic analysis, concept scoring, and personalized feedback generation using Sentence-BERT.

Milestone 3: App.py Development

- **Activity 3.1:** Develop the main application logic in app.py by integrating audio upload, speech recognition, semantic analysis, waveform visualization, and PDF report generation.

Milestone 4: Frontend Development

- **Activity 4.1:** Design a user-friendly interface using Streamlit for audio upload and concept evaluation.
- **Activity 4.2:** Display transcripts, similarity scores, waveform visualization, feedback, and downloadable PDF reports dynamically.

Milestone 5: Testing and Local Deployment

- **Activity 5.1:** Configure the local environment and install all required dependencies.
- **Activity 5.2:** Test speech recognition, semantic analysis, waveform visualization, and PDF report generation locally**.**

Milestone 6: Conclusion

This milestone summarizes the successful development, testing, and evaluation of the Voice-Based Concept Understanding Analyser, demonstrating the integration of AI-based speech recognition, semantic analysis, and automated report generation**.**

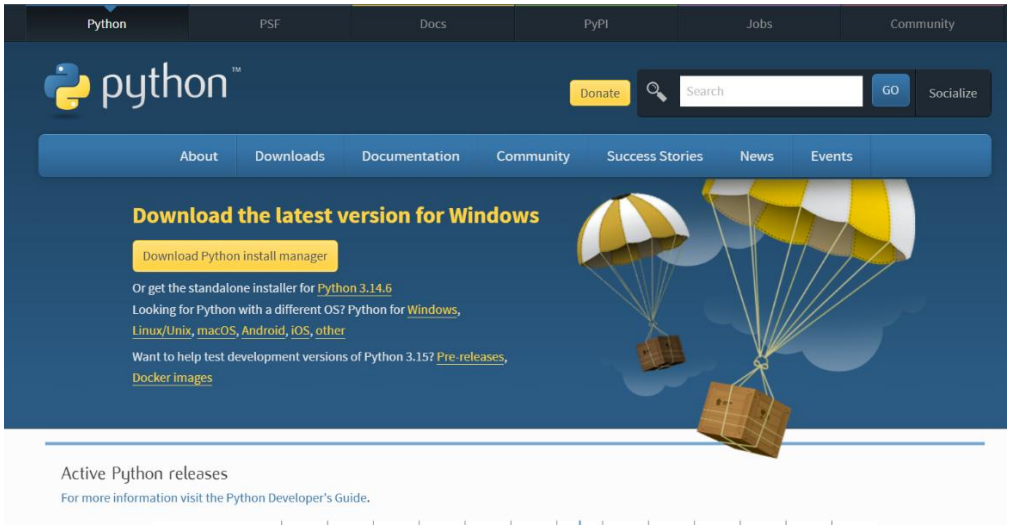
Milestone 1: Speech Recognition Model Selection and System Architecture

In this milestone, the focus is on selecting suitable Artificial Intelligence models and designing the overall architecture of the Voice-Based Concept Understanding Analyser. The system combines speech recognition, semantic similarity analysis, and concept evaluation to assess a user's understanding from spoken explanations. This milestone also includes setting up the development environment and configuring all required tools and libraries.

Activity 1.1: Select and Integrate the OpenAI Whisper Model

The OpenAI Whisper model was selected for speech-to-text conversion because of its high accuracy, multilingual support, and ability to recognize speech in different accents and noisy environments. Whisper converts the uploaded audio into text, which is then used for concept evaluation.
Follow the steps below to integrate the Whisper model into the project.

Step 1 – Install Python
Install Python 3.10 or above from the official Python website and ensure that Python is added to the system PATH during installation.



Step 2 – Install FFmpeg
Install FFmpeg, which is required by the Whisper model to process audio files. After installation, configure the FFmpeg path in the system environment variables.

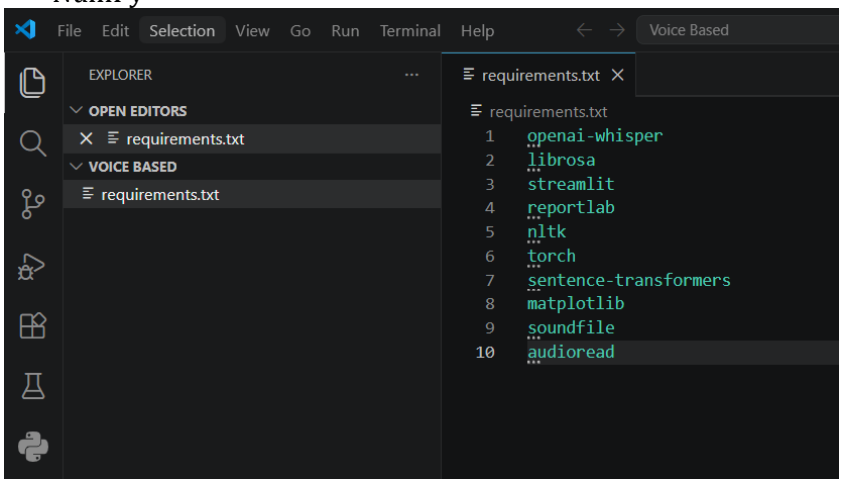
Step 3 – Install Required Libraries

Install all required project dependencies using the following command:

pip install -r requirements.txt

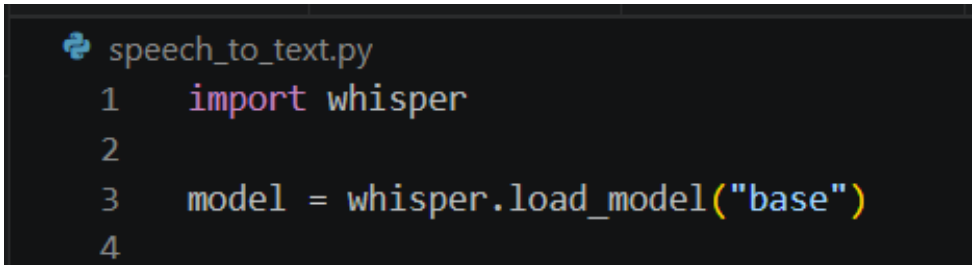
The important libraries include:

- OpenAI Whisper
- Streamlit
- Sentence-BERT
- Librosa
- Matplotlib
- ReportLab
- Torch
- NumPy

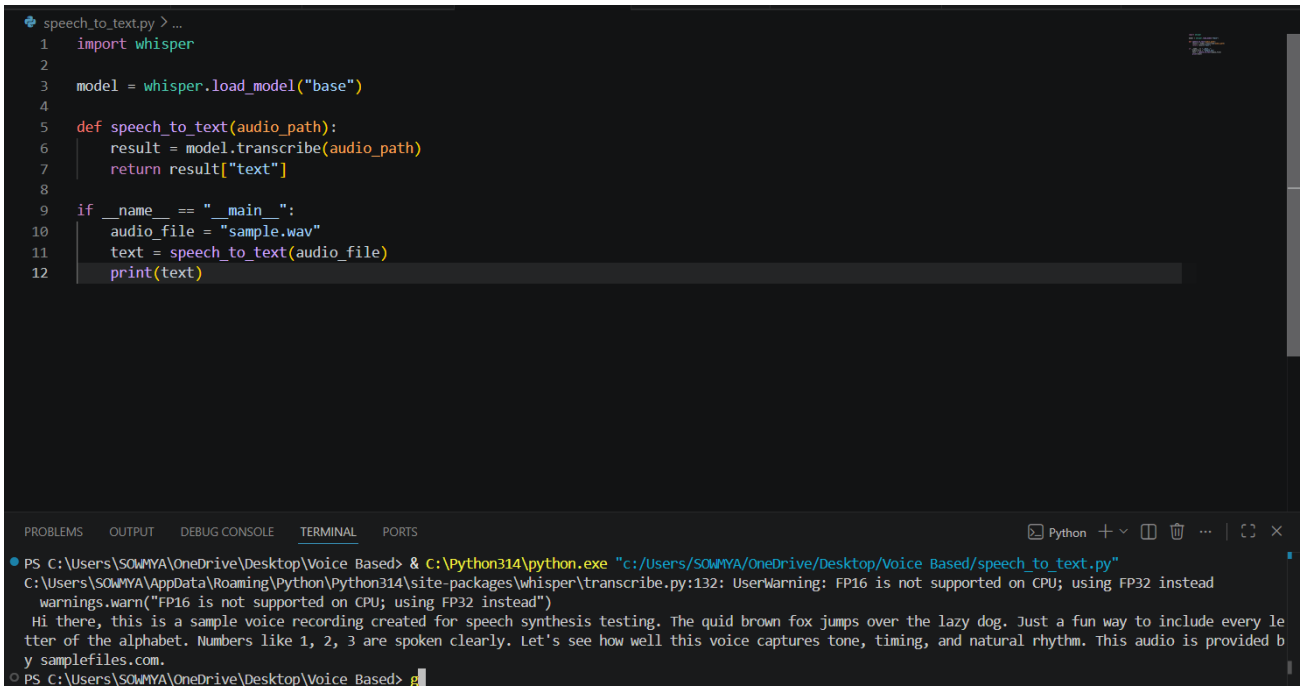


Step 4 – Load the Whisper Model

Load the Whisper model in the application to perform speech-to-text conversion.



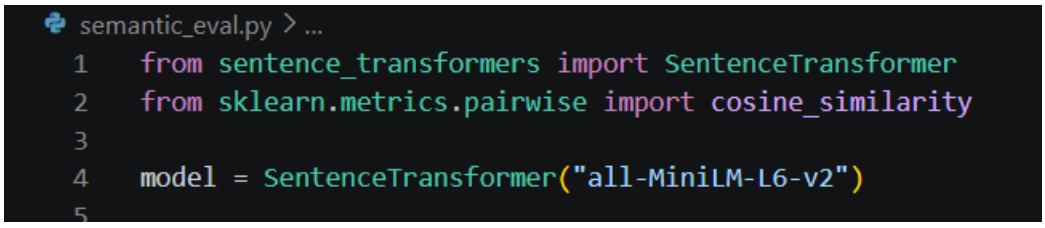
Step 5 – Convert Speech to Text
The uploaded audio is processed by Whisper, which generates an accurate transcript. This transcript is then passed to the semantic analysis module for concept evaluation.



Activity 1.2: Research and Select the Appropriate Semantic Analysis Model

The semantic analysis model plays a crucial role in evaluating the user's conceptual understanding. After researching various Natural Language Processing (NLP) models, Sentence-BERT (all-MiniLM-L6-v2) was selected because it efficiently measures the semantic similarity between the user's spoken explanation and the expected concept.
Here are the factors considered while selecting the model:

- **Understand the Project Requirements:** Review the need to compare the speech transcript with the reference concept accurately.
- **Explore Sentence-BERT Models:** Study the available pre-trained Sentence-BERT models and their semantic similarity capabilities.
- **Evaluate Model Performance:** Compare models based on semantic accuracy, inference speed, and sentence embedding quality.
- **Select the Optimal Model:** Choose all-MiniLM-L6-v2 for its high accuracy, lightweight architecture, and fast semantic similarity computation.



Activity 1.3: Define the System Architecture

The Voice-Based Concept Understanding Analyser follows a modular architecture that integrates speech recognition, semantic analysis, concept evaluation, and report generation. The architecture ensures smooth data flow from audio input to the final evaluation report.

System Architecture Overview
Create a system architecture diagram illustrating the interaction between the Streamlit user interface, audio processing module, OpenAI Whisper, Sentence-BERT, concept scoring engine, waveform visualization, and PDF report generation.

User Interface
Describe how users interact with the application by uploading an audio file through the Streamlit interface. The application displays the generated transcript, similarity score, concept score, personalized feedback, waveform visualization, and provides an option to download the PDF report.

Backend Processing
Explain how the backend receives the uploaded audio, preprocesses it using Librosa, converts speech into text using OpenAI Whisper, performs semantic similarity analysis using Sentence-BERT, calculates the concept understanding score, and prepares the results for display.

AI Model Integration
Describe how OpenAI Whisper performs speech-to-text conversion, Sentence-BERT compares the transcript with the expected concept using semantic similarity, and ReportLab generates the final PDF report containing the evaluation results

Activity 1.4: Set Up the Development Environment

The development environment was configured to support the development of the Voice-Based Concept Understanding Analyser. The required tools and libraries were installed to enable speech recognition, semantic analysis, audio processing, and report generation.

Install Python and Pip

- Install Python 3.10 or above.
- Verify that Python and pip are installed successfully.

```
PS C:\Users\SOWMYA\OneDrive\Desktop\Voice Based> python --version
Python 3.14.4
PS C:\Users\SOWMYA\OneDrive\Desktop\Voice Based> pip --version
pip 26.1.1 from C:\Users\SOWMYA\AppData\Roaming\Python\Python314\site-packages\pip (python 3.14)
```

Install Project Dependencies
Install all the required libraries using the following command:
pip install -r requirements.txt

```
PS C:\Users\SOWMYA\OneDrive\Desktop\Voice Based> pip install -r requirements.txt
```

Install FFmpeg

Install FFmpeg and configure it to enable audio processing for the Whisper speech recognition model.

Run the Application

Start the application using the following command:

streamlit run app.py

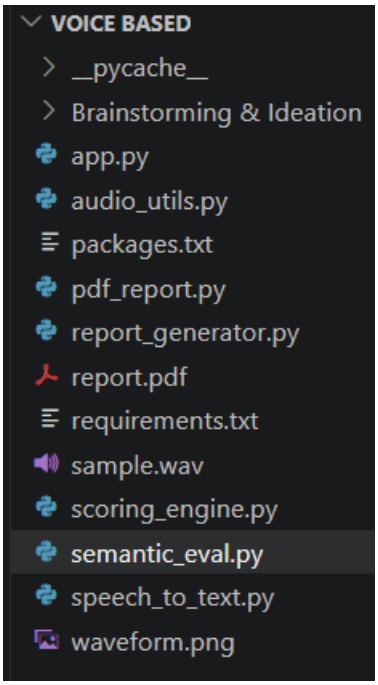
```
PS C:\Users\SOWMYA\OneDrive\Desktop\Voice Based> streamlit run app.py
```

The application launches in the default web browser, allowing users to upload audio files, analyze concept understanding, and generate PDF reports.

Set Up the Project Structure

Organize the project files, including:

- app.py
- speech_to_text.py
- semantic_eval.py
- audio_utils.py
- scoring_engine.py
- report_generator.py
- pdf_report.py
- requirements.txt



Milestone 2: Core Functionalities Development

This milestone focuses on developing the core functionalities of the Voice-Based Concept Understanding Analyser. The application processes the user's speech, converts it into text, evaluates conceptual understanding using semantic similarity, and generates feedback along with a detailed PDF report.

Activity 2.1: Develop the Speech-to-Text Module

The Speech-to-Text module is the foundation of the application. It converts the uploaded voice recording into text using the OpenAI Whisper model, allowing the system to analyze the user's conceptual explanation.

Accept Audio Input

- Allow users to upload a .wav audio file through the Streamlit interface.
- Validate the uploaded audio before processing.

Process Audio

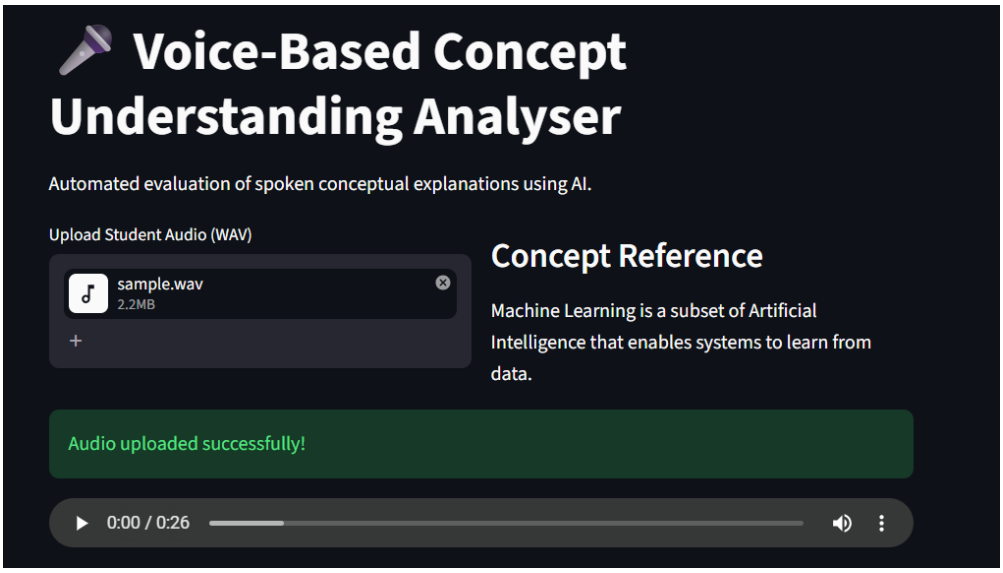
- Read and preprocess the uploaded audio file.
- Prepare the audio for speech recognition.

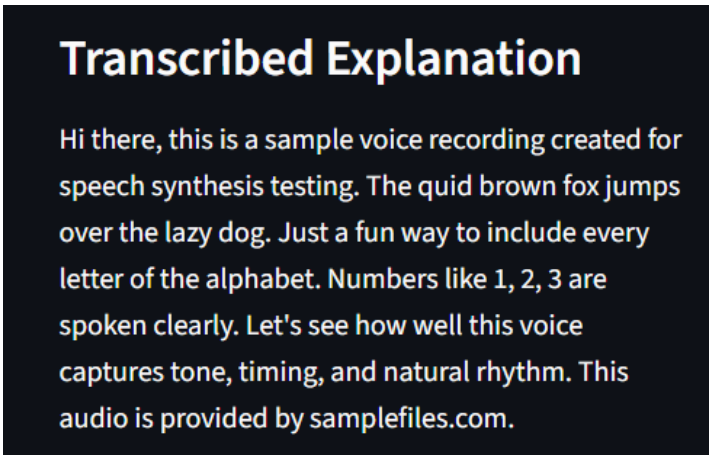
Perform Speech-to-Text Conversion

- Load the OpenAI Whisper model.
- Convert the speech into an accurate text transcript.

Display the Transcript

- Display the generated transcript on the Streamlit interface.
- Pass the transcript to the semantic analysis module for further evaluation.





Activity 2.2: Develop the Semantic Analysis and Concept Evaluation Module

The semantic analysis module evaluates the user's conceptual understanding by comparing the generated transcript with the reference concept. The application uses Sentence-BERT to measure semantic similarity and generates the final concept evaluation based on the analysis results.

Perform Semantic Analysis

- Generate sentence embeddings for the transcript and reference concept using Sentence-BERT.
- Calculate the semantic similarity using Cosine Similarity.

Evaluate Concept Understanding

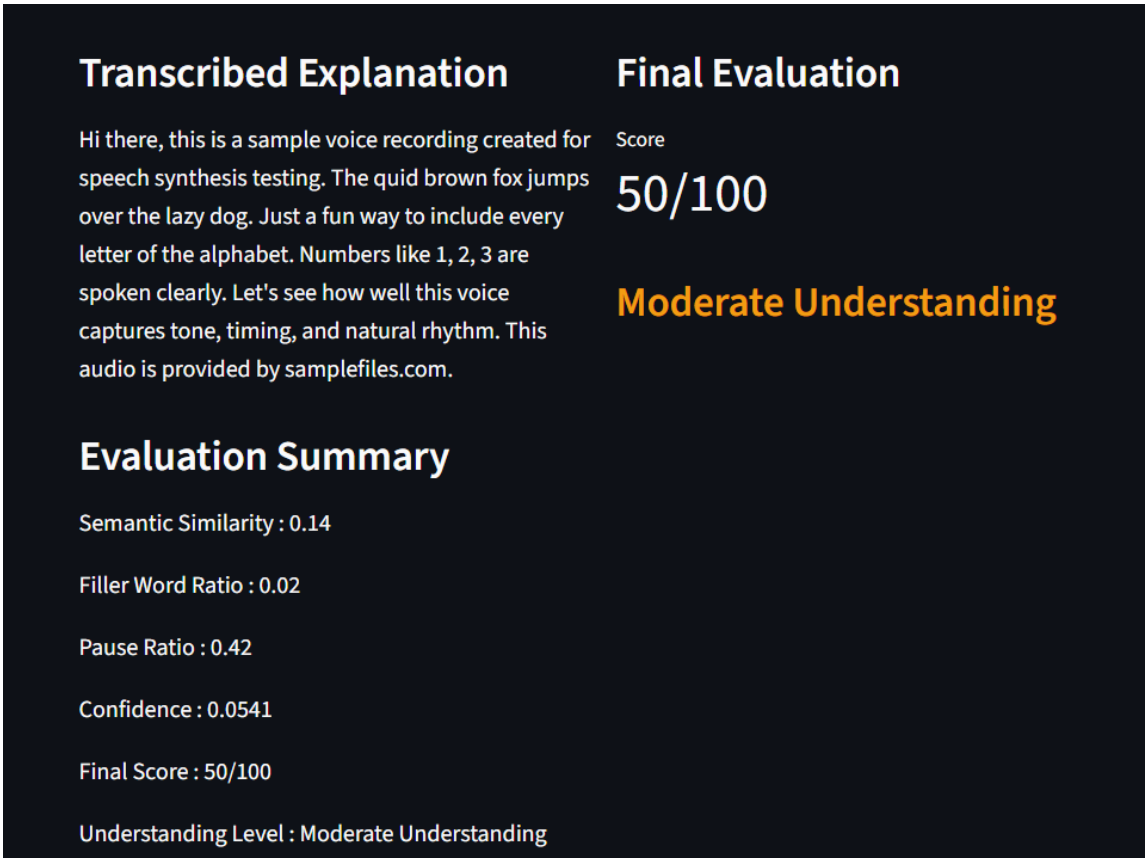
- Calculate the Final Score based on semantic similarity and speech analysis.
- Determine the user's Understanding Level and generate an evaluation summary.

Generate Audio Waveform

- Process the uploaded audio using Librosa.
- Generate and display the audio waveform using Matplotlib to provide a visual representation of the recorded speech.

Display the Evaluation Results

- Display the Semantic Similarity Score, Final Score, Understanding Level, Evaluation Summary, and Audio Waveform on the Streamlit interface.
- Generate the evaluation report in PDF format for download.



Milestone 3: App.py Development

Milestone 3 focuses on developing the app.py file, which serves as the main entry point of the Voice-Based Concept Understanding Analyser. It integrates all project modules and manages the complete workflow from audio upload to concept evaluation and report generation.

Activity 3.1: Develop the Main Application Logic in app.py

Import Required Modules

- Import the required Python libraries and project modules.
- Integrate modules for speech recognition, semantic analysis, audio processing, concept scoring, and PDF report generation.

Configure the Streamlit Interface

- Configure the application title and page settings.
- Create a user-friendly interface for uploading audio files.

Process User Input

- Accept audio files uploaded by the user.
- Invoke the speech recognition, semantic analysis, and scoring modules.
- Display the evaluation results and generate the PDF report.

app.py > ...

```python
import streamlit as st
import librosa
import matplotlib.pyplot as plt
import tempfile
from pdf_report import create_pdf
from speech_to_text import speech_to_text

from semantic_eval import semantic_similarity
from audio_utils import analyze_audio
from scoring_engine import evaluate_understanding

st.set_page_config(page_title="Voice-Based Concept Understanding Analyser")

st.title("🎤 Voice-Based Concept Understanding Analyser")
st.write("Automated evaluation of spoken conceptual explanations using AI.")

col1, col2 = st.columns(2)

with col1:
    uploaded_file = st.file_uploader(
        "Upload Student Audio (WAV)",
        type=["wav", "mp3"]
    )
```

4: Frontend Development

Introduction

Milestone 4 focuses on designing an intuitive and interactive user interface for the Voice-Based Concept Understanding Analyser. The frontend is developed using Streamlit, providing users with a simple interface to upload audio files, analyze conceptual understanding, and view evaluation results.

Activity 4.1: Designing the User Interface

Create the User Interface

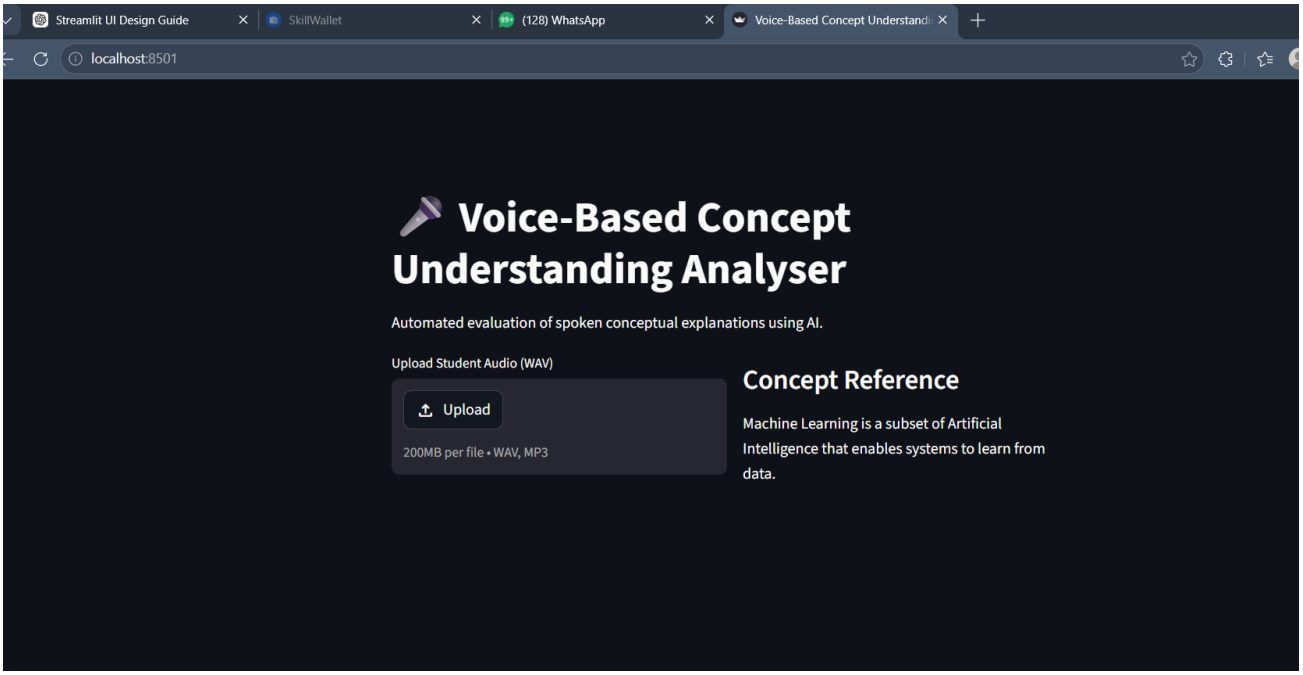
- Design the application using Streamlit with a clean and user-friendly layout.
- Display the project title and a brief description of the application.

Create the Audio Upload Section

- Provide an interface for users to upload WAV or MP3 audio files.
- Display the selected audio file before analysis.

Display the Reference Concept

- Show the reference concept that will be used for semantic comparison.
- Ensure the information is clearly visible to the user.



Activity 4.2: Displaying the Evaluation Results

The application dynamically displays the evaluation results after processing the uploaded audio. The interface presents the transcript, concept evaluation, and overall performance in a structured and easy-to-understand format.

Display the Speech Transcript

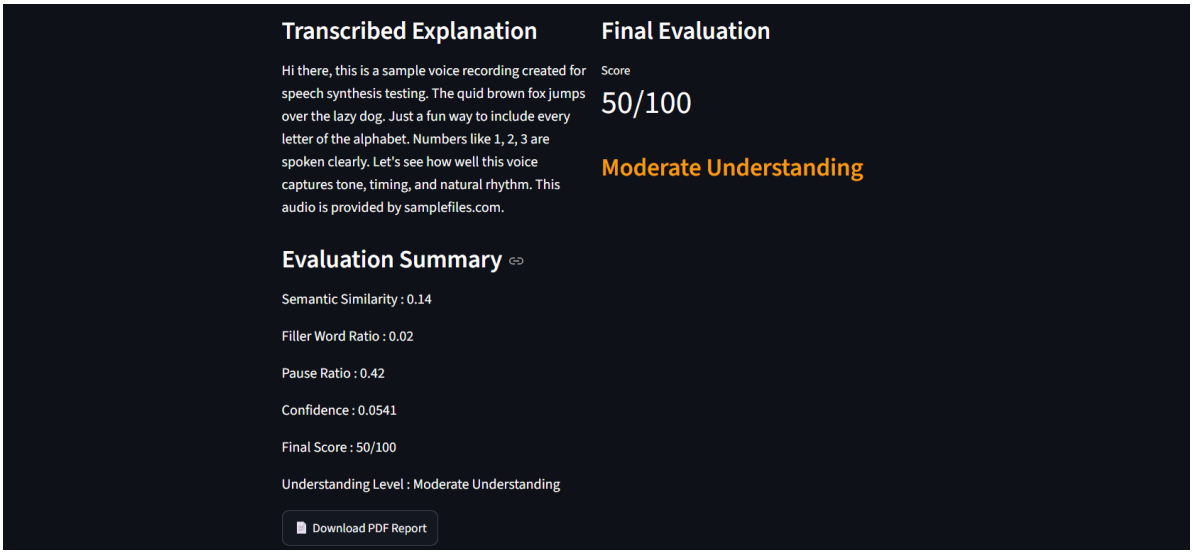
- Display the transcribed explanation generated by the OpenAI Whisper model.
- Present the transcript in a readable format for user verification.

Display the Evaluation Summary

- Display the Semantic Similarity Score.
- Display the Final Evaluation Score.
- Display the Understanding Level based on the evaluation.

Display the Final Results

- Present the complete evaluation summary.
- Allow users to review their performance before downloading the generated PDF report.



Milestone 5: Testing and Local Deployment

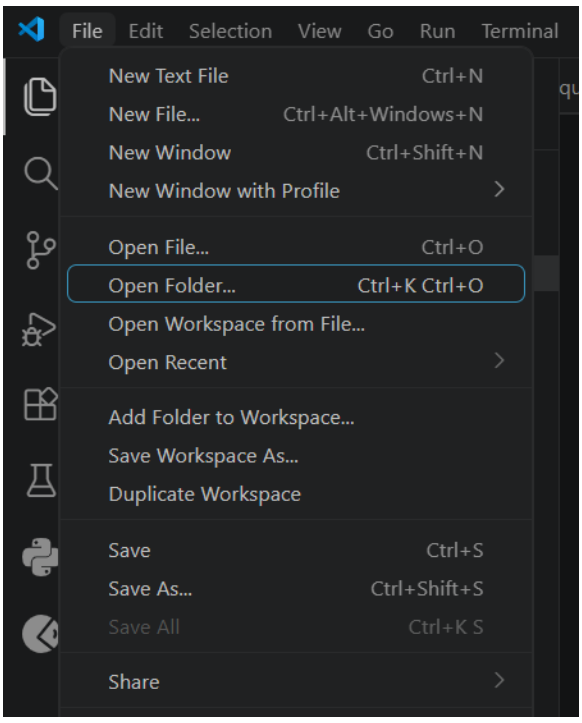
In this milestone, the focus is on running the Voice-Based Concept Understanding Analyser in the local development environment and verifying that all functionalities work correctly. The application is executed locally using Streamlit and accessed through the localhost URL.

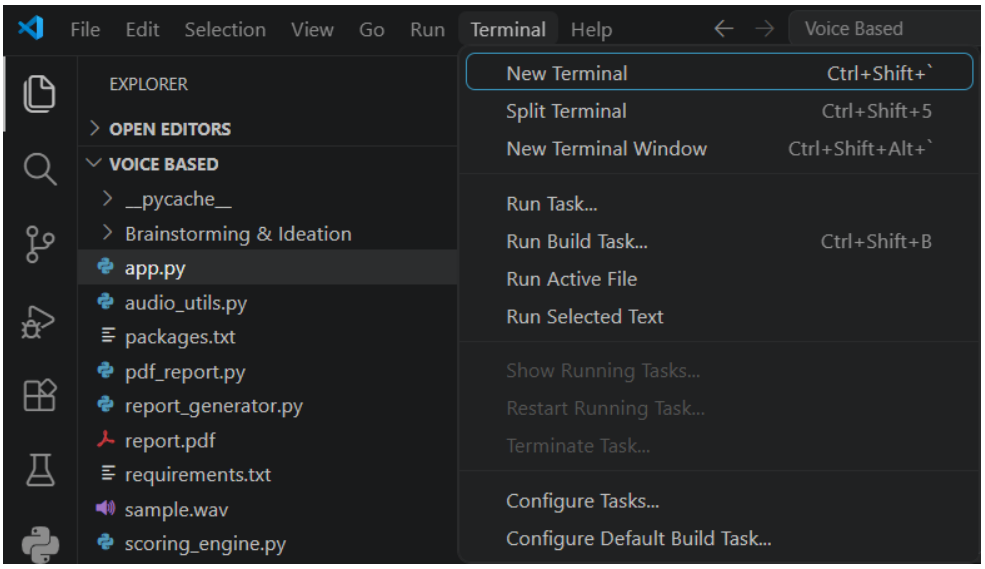
Activity 5.1: Preparing the Application for Local Deployment

To execute the Voice-Based Concept Understanding Analyser locally, the project environment must be configured and all required dependencies should be installed before launching the application.

Open the Project Directory

- Open the project folder in Visual Studio Code.
- Open the integrated terminal from the Terminal menu.





Install Project Dependencies

Install all required Python libraries using the following command:

pip install -r requirements.txt

```
PS C:\Users\SOWMYA\OneDrive\Desktop\Voice Based> pip install -r requirements.txt
```

Run the Streamlit Application

Start the application by executing the following command:

streamlit run app.py

```
PS C:\Users\SOWMYA\OneDrive\Desktop\Voice Based> streamlit run app.py
```

Access the Application

After the application starts successfully, Streamlit generates a local URL in the terminal similar to the following:

Local URL: http://localhost:8501

Open the above URL in any web browser to launch the Voice-Based Concept Understanding Analyser.

```
PS C:\Users\SOWMYA\OneDrive\Desktop\Voice Based> python -m streamlit run app.py
  2026-06-30 13:35:08.744 Uvicorn server started on 0.0.0.0:8501

    You can now view your Streamlit app in your browser.

    Local URL: http://localhost:8501
    Network URL: http://192.168.8.41:8501
```

Verify Application Startup

- Confirm that the Streamlit home page loads successfully.
- Verify that the audio upload option is displayed.
- Ensure the application is ready to accept audio files for concept analysis.

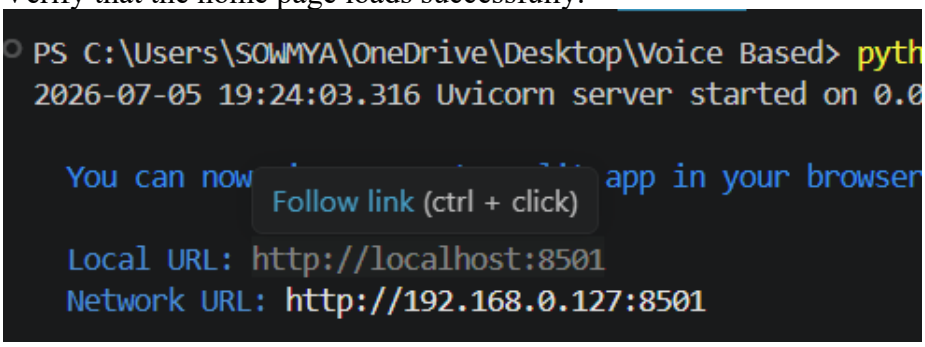
Activity 5.2: Local Testing and Verification

After successfully launching the application on the local host, comprehensive testing was performed to verify that each module functions correctly.

Verify Application Access

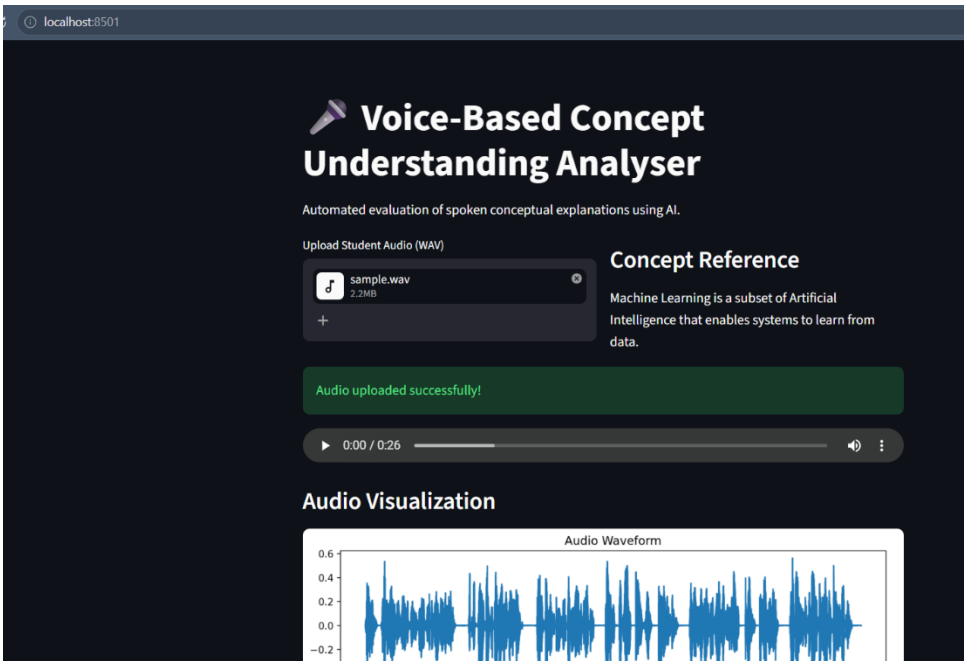
- Open the application using the local URL:
- http://localhost:8501

- Verify that the home page loads successfully.



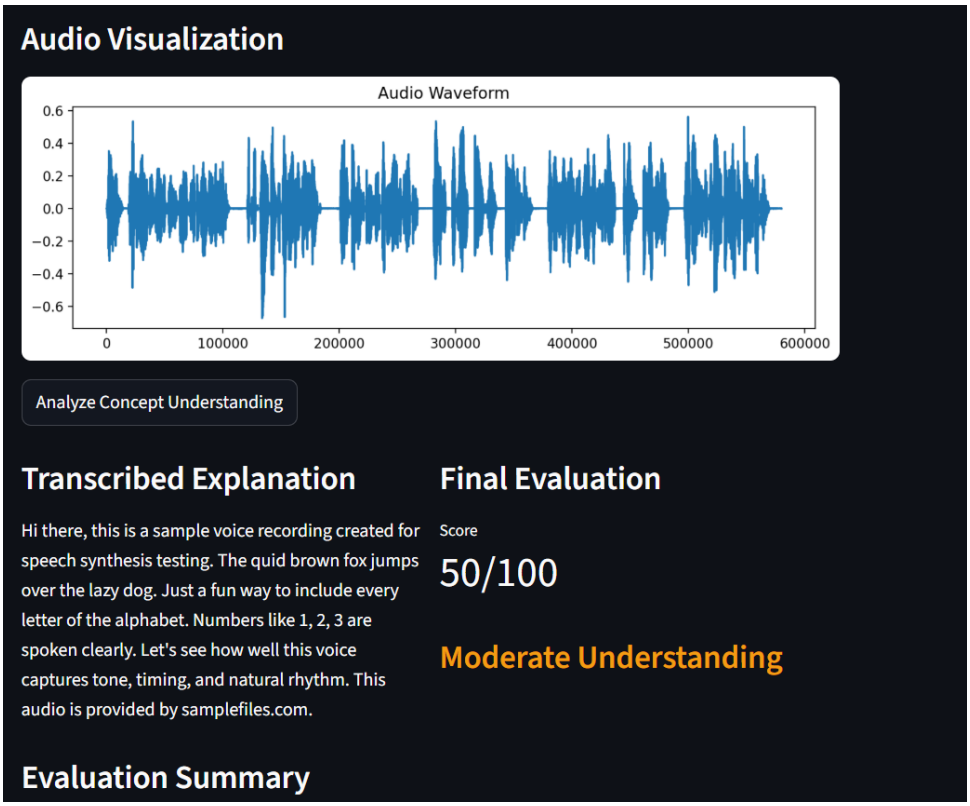
Test Audio Upload

- Upload a sample .wav audio file using the Streamlit interface.
- Ensure the uploaded audio is accepted and displayed successfully.



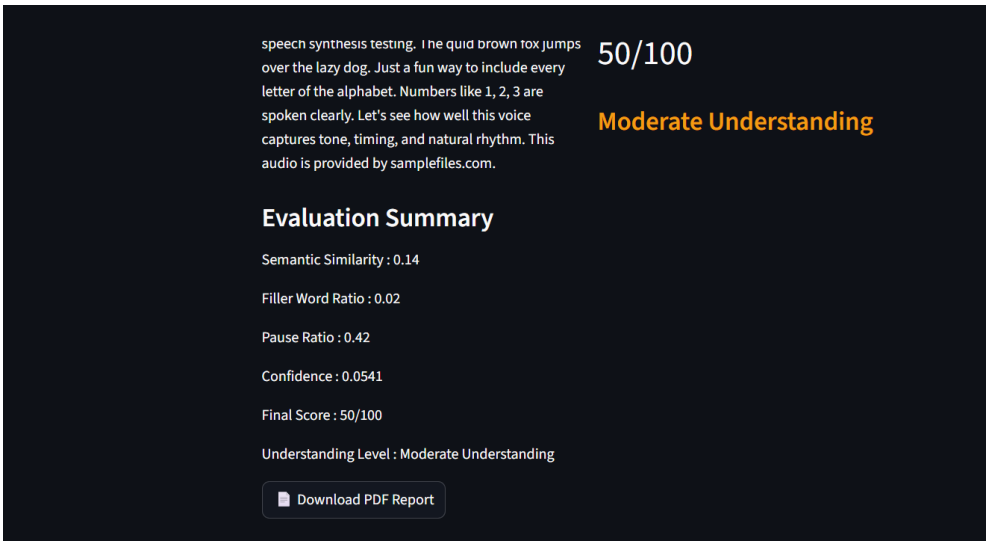
Test Speech-to-Text Conversion

- Verify that the OpenAI Whisper model converts the uploaded speech into an accurate text transcript.



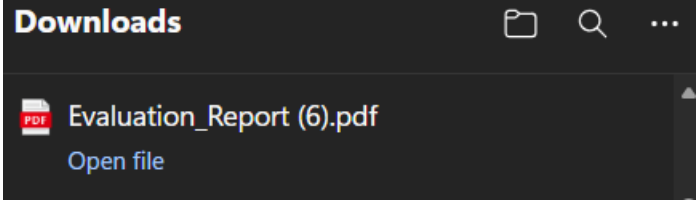
Test Semantic Analysis and Concept Evaluation

- Verify that the application calculates the Semantic Similarity Score using Sentence-BERT.
- Confirm that the Final Score, Understanding Level, and Evaluation Summary are displayed correctly.



Test PDF Report Generation

- Generate the PDF report.
- Verify that the report is created successfully and is available for download.



Voice-Based Concept Understanding Report

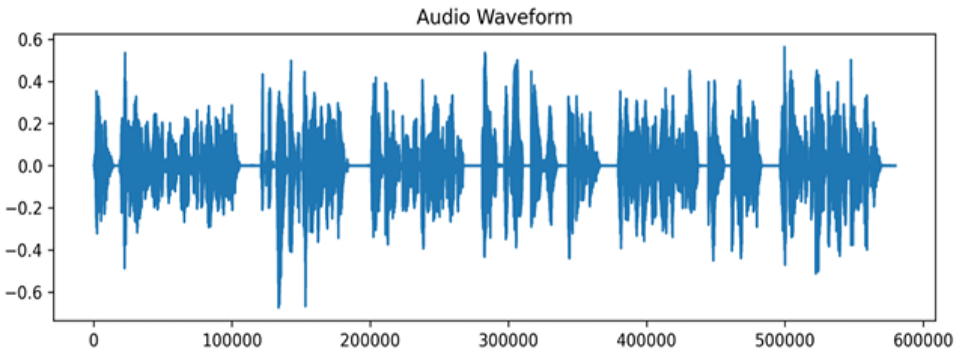
Reference Concept

Machine Learning is a subset of Artificial Intelligence that enables systems to learn from data.

Student Transcription

Hi there, this is a sample voice recording created for speech synthesis testing. The quid brown fox jumps over the lazy dog. Just a fun way to include every letter of the alphabet. Numbers like 1, 2, 3 are spoken clearly. Let's see how well this voice captures tone, timing, and natural rhythm. This audio is provided by samplefiles.com.

Audio Visualization



Evaluation Summary

Metric	Value
Semantic Similarity	0.14
Filler Word Ratio	0.02
Pause Ratio	0.42
Confidence (Energy)	0.0541
Final Score	50/100
Understanding Level	Moderate Understanding

Verify Overall Application Performance

- Ensure all modules execute without errors.
- Confirm that the application responds correctly to user interactions and displays all evaluation results successfully.

Milestone 6: Conclusion

The Voice-Based Concept Understanding Analyser successfully integrates Speech Recognition, Natural Language Processing, Semantic Analysis, Audio Waveform Visualization, and PDF Report Generation into a single intelligent application. By converting spoken explanations into text using OpenAI Whisper and evaluating conceptual understanding through Sentence-BERT, the application provides meaningful insights into a user's knowledge of a given topic.

The system features an interactive Streamlit interface that allows users to upload audio, visualize the speech waveform, view transcripts, analyze concept understanding, and download a detailed evaluation report. The application was successfully tested in a local development environment, demonstrating reliable performance across all core functionalities.

This project highlights the practical use of Artificial Intelligence in education and self-learning by helping students and learners assess their conceptual understanding through voice-based analysis. In the future, the application can be enhanced with real-time speech analysis, multilingual support, cloud deployment, advanced feedback mechanisms, and user authentication to further improve its usability and scalability.